

API-Typen und ihre Anwendungsfälle

Eine graphentheoretische Analyse mittels Subgraph Algorithmus

Stephan Epp

25. Juni 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Problemstellung	3
1.2	Motivation	3
1.3	Beiträge der Arbeit	4
2	Grundlagen	4
2.1	Graphentheoretische Grundlagen	4
2.2	API-Graphen	5
2.3	API-Taxonomie	5
3	API-Protokolle: Formale Analyse	6
3.1	REST-API	6
3.2	SOAP-API	6
3.3	GraphQL-API	7
3.4	B2B-Integration	7
3.5	Vergleichende Analyse	8
4	Sequenzdiagramme der API-Protokolle	8
4.1	REST-API: Sequenzdiagramm	8
4.2	SOAP-API: Sequenzdiagramm	8
4.3	GraphQL-API: Sequenzdiagramm	8
4.4	B2B-Integration: Sequenzdiagramm	9
4.5	Service-zu-Datenbank: Ablaufdiagramm	9
5	Subgraph Algorithmus auf API-Graphen	9
5.1	Grundprinzip und Überblick	9
5.2	Signaturberechnung für API-Graphen	10
5.3	Zyklische Rotation und LCS-Matching	11
5.4	Korrektheit	12
5.5	Laufzeitkomplexität	12
6	API-Sicherheitsanalyse mit dem Subgraph Algorithmus	13
6.1	Angriffsmuster als Subgraphen	13

7	Matplotlib-Visualisierungen	14
7.1	API-Taxonomie-Baum	14
7.2	Subgraph-Matching auf API-Graphen	15
7.3	Komplexitätsanalyse und LCS-Matrix	15
7.4	Zyklische Rotation und Signaturen	15
7.5	Relevanz-Heatmap und Eigenschaftsprofil	16
8	Zusammenfassung	16
9	Ausblick	17
9.1	GraphQL-Schema-Optimierung und Federation-Analyse	17
9.2	API-Gateway-Verifikation und Konfigurationsüberprüfung	18
9.3	KI-gestützte API-Erkennung und Klassifikation	19
9.4	Semantische API-Versionierung mit dem Subgraph Algorithmus	19
9.5	Gewichtete API-Graphen: Leistungs- und Kostenanalyse	20
9.6	Parallele und verteilte Subgraph-Berechnung	21
9.7	Formale Standardisierung und API-Marktplätze	21

1 Einführung

Application Programming Interfaces (APIs) bilden das Rückgrat moderner verteilter Softwarearchitekturen. Sie definieren standardisierte Kommunikationsverträge zwischen heterogenen Systemkomponenten und ermöglichen damit eine lose Kopplung, die für skalierbare, wartbare und interoperable Systeme unabdingbar ist [1]. Ohne APIs wäre die heutige Form des Internets — als ein Geflecht aus Milliarden von miteinander kommunizierenden Diensten — nicht denkbar. Sie sind das universelle Bindeglied zwischen Front-Ends und Back-Ends, zwischen Unternehmenssystemen und der Cloud, zwischen menschlichen Anwendern und maschinellen Prozessen.

Die Vielzahl existierender API-Stile, -Protokolle und -Integrationsmuster hat in der Praxis jedoch zu einer erheblichen Komplexität geführt. Entwicklerinnen und Entwickler stehen vor der Frage, welches API-Paradigma für einen gegebenen Anwendungsfall am geeignetsten ist, wie vorhandene API-Architekturen strukturell verglichen werden können und wie Redundanzen in gewachsenen API-Ökosystemen erkannt und beseitigt werden können.

Die vorliegende Arbeit begegnet diesen Fragen mit einem formalen Ansatz: API-Kommunikationsstrukturen werden als gerichtete Graphen modelliert. Der *Subgraph Algorithmus* [4] — ein $O(n^3)$ -Verfahren zur strukturellen Graphanalyse mittels injektiver Signaturen und zyklischer LCS-Rotation — wird auf diese API-Graphen angewendet und liefert effiziente, maschinell nachvollziehbare Antworten auf die genannten Fragen.

1.1 Problemstellung

Gegeben sei eine Menge von API-Endpunkten $\mathcal{E} = \{e_1, \dots, e_n\}$ und ihrer Abhängigkeiten $\mathcal{R} \subseteq \mathcal{E} \times \mathcal{E}$. Gesucht ist eine formale Methode, die folgende Fragen polynomiell in $O(n^3)$ beantwortet:

- **Strukturelle Inklusion:** Ist eine API-Architektur A als Subgraph in einer umfassenderen Architektur B enthalten?
- **Redundanzerkennung:** Existieren redundante API-Ketten, die ohne Informationsverlust dedupliziert werden können?
- **Protokollklassifikation:** Wie lassen sich verschiedene API-Protokolle (REST, SOAP, GraphQL) formal klassifizieren, verglichen und in ein einheitliches Modell überführt werden?
- **Kompatibilitätsprüfung:** Ist eine neue API-Version rückwärtskompatibel zu einer bestehenden?
- **Sicherheitsanalyse:** Enthält ein API-Graph eine bekannte Angriffssignatur als Subgraph?

1.2 Motivation

In der Praxis wachsen API-Ökosysteme organisch und unkontrolliert. Ohne formale Strukturanalyse entstehen redundante Endpunkte, inkonsistente Versionierungsstrategien und schwer nachvollziehbare Abhängigkeitsgeflechte. Studien zeigen, dass in großen Unternehmen durchschnittlich mehrere hundert interne APIs existieren, von denen ein erheblicher

Anteil funktionale Überschneidungen aufweist [8]. Die manuelle Analyse solcher Strukturen ist fehleranfällig und skaliert nicht.

Eine graphentheoretische Modellierung, kombiniert mit dem Subgraph Algorithmus [4], ermöglicht eine systematische, maschinell prüfbare Analyse. Der Algorithmus benötigt lediglich die Adjazenzmatrizen der API-Graphen als Eingabe und liefert in $O(n^3)$ eine eindeutige Entscheidung über strukturelle Inklusion — eine erhebliche Verbesserung gegenüber naiven Ansätzen mit $O(n! \cdot n^2)$ Laufzeit.

1.3 Beiträge der Arbeit

Die zentralen Beiträge dieser Arbeit sind:

1. Formale Definition einer vollständigen API-Taxonomie als gerichteter Graph mit drei Hierarchieebenen (Kategorie, Protokoll, Anwendungsfall).
2. Nachweis, dass der Subgraph Algorithmus auf API-Graphen korrekt und in $\Theta(n^3)$ anwendbar ist, einschließlich Vollständigkeits- und Korrektheitsbeweis.
3. Formaler Beweis der Optimalität unter der Strong Exponential Time Hypothesis.
4. Entwicklung von fünf TikZ-Sequenz- und Ablaufdiagrammen für alle wesentlichen API-Protokolle (REST, SOAP, GraphQL, B2B, Service-zu-Datenbank).
5. Quantitative Auswertung von Komplexität, Relevanz und Eigenschaftsprofilen der API-Typen mittels fünf Matplotlib-Visualisierungen.
6. Ausführliche formale Analyse von API-Sicherheitsaspekten, Versionierungsstrategien und Erweiterungen auf gewichtete Graphen.
7. Sehr ausführlicher Ausblick auf zukünftige Forschungsrichtungen.

2 Grundlagen

2.1 Graphentheoretische Grundlagen

Definition 1 (Gerichteter Graph). Ein *gerichteter Graph* $G = (V, E)$ besteht aus einer endlichen Knotenmenge $V = \{v_1, \dots, v_n\}$ und einer Kantenmenge $E \subseteq V \times V$. Eine Kante $(v_i, v_j) \in E$ repräsentiert einen gerichteten Kommunikationskanal von Knoten v_i zu Knoten v_j .

Definition 2 (Adjazenzmatrix). Die *Adjazenzmatrix* $A \in \{0, 1\}^{n \times n}$ eines gerichteten Graphen $G = (V, E)$ mit $|V| = n$ ist definiert durch:

$$A_{ij} = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E, \\ 0 & \text{sonst.} \end{cases}$$

Definition 3 (Subgraph). Ein gerichteter Graph $G = (V, E)$ ist ein *Subgraph* von $G' = (V', E')$, geschrieben $G \subseteq G'$, wenn $V \subseteq V'$ und $E \subseteq E'$.

Definition 4 (Längste gemeinsame Teilsequenz). Seien $\sigma^A = [\sigma_0^A, \dots, \sigma_{m-1}^A]$ und $\sigma^B = [\sigma_0^B, \dots, \sigma_{k-1}^B]$ zwei Sequenzen. Die *längste gemeinsame Teilsequenz* (LCS) ist die längste Sequenz σ^C , die sowohl Teilsequenz von σ^A als auch von σ^B ist. Die Länge der LCS wird mit $\text{LCS}(\sigma^A, \sigma^B)$ bezeichnet und in $O(m \cdot k)$ mittels dynamischer Programmierung berechnet.

Definition 5 (Zyklische Rotation). Die k -te *zyklische Rotation* einer Sequenz $\sigma = [\sigma_0, \dots, \sigma_{n-1}]$ ist:

$$\text{rot}_k(\sigma) = [\sigma_k, \sigma_{k+1}, \dots, \sigma_{n-1}, \sigma_0, \dots, \sigma_{k-1}].$$

2.2 API-Graphen

Definition 6 (API-Graph). Sei $\mathcal{E} = \{e_1, \dots, e_n\}$ eine Menge von API-Endpunkten und $\mathcal{R} \subseteq \mathcal{E} \times \mathcal{E}$ eine gerichtete Abhängigkeitsrelation. Der *API-Graph* ist der gerichtete Graph $G_{\text{API}} = (\mathcal{E}, \mathcal{R})$.

Definition 7 (API-Typ). Ein *API-Typ* T ist ein Tupel $T = (\Pi, \mathcal{S}, \mathcal{U}, \mathcal{C})$, bestehend aus einem Protokoll Π , einer Menge von Sicherheitseigenschaften $\mathcal{S}$, einer Menge von Anwendungsfällen $\mathcal{U}$ und einer Menge von Kompatibilitätsbedingungen $\mathcal{C}$.

Definition 8 (Injektive Signatur-Funktion). Sei $A \in \{0, 1\}^{n \times n}$ eine Adjazenzmatrix. Die *injektive Signatur-Funktion* $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist definiert durch:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

2.3 API-Taxonomie

Die in dieser Arbeit betrachtete Taxonomie gliedert sich in drei Hauptkategorien, die sich jeweils weiter in Protokolle und Anwendungsfälle unterteilen:

- **Open API:** Öffentlich zugängliche Schnittstellen, typischerweise über REST, SOAP oder GraphQL realisiert. Sie richten sich an eine breite Entwicklercommunity und sind über öffentliche Dokumentationsportale beschrieben. Anwendungsfälle umfassen Wetterdaten, Login-Systeme, Produktabfragen, Banktransfers, Versicherungsansprüche und Behördendaten.
- **Internal API:** Interne Schnittstellen zwischen Systemkomponenten, die ausschließlich innerhalb der Systemgrenzen einer Organisation genutzt werden. Untertypen sind Backend-zu-Backend, Frontend-zu-Backend und Service-zu-Datenbank. Anwendungsfälle umfassen Benutzerauthentifizierung, Profilabruf, Live-Suche, Benutzerverwaltung, Profilaktualisierung und Berichtsabfragen.
- **Partner API:** Schnittstellen für kontrollierte externe Zugriffe durch Geschäftspartner, mit expliziten Zugangsbeschränkungen und SLA-Vereinbarungen. Untertypen sind B2B-Integration, Affiliate-Integration und Data-Sharing-API. Anwendungsfälle umfassen Hotelbuchung, Flugdaten, Zahlungsgateway, Produktlinks, Provisionsverfolgung, Klickanalysen, Gesundheitsdaten, Finanzdaten und Logistikdaten.

3 API-Protokolle: Formale Analyse

3.1 REST-API

Definition 9 (REST-API). Eine *Representational State Transfer API* (REST-API) ist eine zustandslose Client-Server-Architektur, bei der Ressourcen durch eindeutige URIs identifiziert und über standardisierte HTTP-Methoden {GET, POST, PUT, DELETE, PATCH} manipuliert werden [1]. REST ist kein Protokoll, sondern ein Architekturstil, der sechs Constraints definiert: Client-Server-Trennung, Zustandslosigkeit, Cachebarkeit, einheitliche Schnittstelle, geschichtetes System und optionaler Code-on-Demand.

Satz 1 (REST-Graph-Modellierung). Eine REST-API mit n Endpunkten lässt sich als gerichteter Graph $G_{\text{REST}} = (V, E)$ mit $|V| = n$ und $|E| = O(n^2)$ modellieren.

Beweis. Jeder Endpunkt $e_i \in V$ repräsentiert eine Ressource. Jede HTTP-Methode erzeugt eine gerichtete Kante (e_i, e_j) , wenn Methode m auf e_i zu einer Zustandsänderung in e_j führt (z. B. POST /orders führt zu einer Aktualisierung von /inventory). Da jeder der n Endpunkte mit maximal $n - 1$ anderen verbunden sein kann, gilt $|E| \leq n(n - 1) = O(n^2)$. Die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ repräsentiert diese Struktur vollständig in $O(n^2)$ Speicher. $\square$

Proposition 1 (Zustandslosigkeit und Graphstruktur). Die Zustandslosigkeit von REST impliziert, dass jede Kante $(e_i, e_j) \in E$ die vollständige Anfrageinformation trägt und keine impliziten Zustandsübergänge zwischen Kanten existieren. Der API-Graph ist damit ein *zustandsloser gerichteter Graph*, bei dem die Kantengewichte durch die HTTP-Methoden und Ressourcenrepräsentationen vollständig bestimmt sind.

Beispiel 1 (REST-Anwendungsfall: Wetterdaten). Sei $V = \{\text{Client, Auth, WeatherSvc, DB}\}$ mit den Endpunkten /auth, /weather, /weather/{city}, /history. Die Adjazenzmatrix lautet:

$$A_{\text{REST}} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Der Client sendet GET-Anfragen an den Auth- und Weather-Service; beide greifen auf die Datenbank zu ($A_{14} = A_{24} = 1$, nullindiziert $A_{13} = A_{23} = 1$).

3.2 SOAP-API

Definition 10 (SOAP-API). Eine *Simple Object Access Protocol API* (SOAP-API) ist ein XML-basiertes Nachrichtenprotokoll nach dem W3C-Standard [2]. Jede SOAP-Nachricht besteht aus einem *Envelope* (Rahmen), einem optionalen *Header* (Metadaten, Sicherheitstoken) und einem *Body* (Nutzlast). SOAP-Dienste werden durch WSDL-Dokumente (Web Services Description Language) formal spezifiziert.

Satz 2 (SOAP-Verbindlichkeit). Für jeden SOAP-Aufruf $(e_i, e_j) \in E_{\text{SOAP}}$ existiert ein formaler WSDL-Kontrakt ω_{ij} , der Eingabeparameter, Ausgabeparameter und Fehlercodes vollständig und maschinenlesbar spezifiziert.

Beweis. Die Web Services Description Language (WSDL) ist eine XML-Grammatik mit eigenem W3C-Schema. Eine gültige SOAP-Implementierung muss für jeden Dienst ein WSDL-Dokument bereitstellen, das die Operationen, Nachrichten, Port-Typen und Bindungen vollständig definiert. Die formale Verifikation erfolgt durch Schema-Validierung des WSDL-Dokuments gegen das W3C-Schema, was eine eindeutige, injektive Abbildung $\omega : E_{\text{SOAP}} \rightarrow \text{WSDL}$ definiert. $\square$

Beispiel 2 (SOAP: Banktransfer). $V = \{\text{Client}, \text{SOAPGateway}, \text{LimitCheck}, \text{DB}\}$.

$$A_{\text{SOAP}} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Der Client sendet einen **TransferRequest**; das Gateway prüft das Limit und schreibt die Transaktion atomar in die Datenbank.

3.3 GraphQL-API

Definition 11 (GraphQL-API). Eine *GraphQL-API* ist eine Abfragesprache für APIs und eine serverseitige Laufzeitumgebung, die dem Client ermöglicht, exakt die benötigten Datenfelder zu spezifizieren [3]. GraphQL definiert ein typisiertes Schema $\mathcal{S}$ mit Typen, Feldern und Resolver-Funktionen $\rho : \mathcal{S} \times \text{Context} \rightarrow \mathcal{D}$. Mutationen verändern Daten; Subscriptions erlauben Echtzeit-Push-Kommunikation.

Satz 3 (GraphQL-Flexibilitätssatz). Eine GraphQL-API mit Schema $\mathcal{S}$ erzeugt für eine Query der Tiefe d einen API-Graphen G_{GQL} mit $|E| \leq |\mathcal{S}|^d$. Mit Query-Tiefenlimit $d_{\max} = O(\log n)$ gilt $|E| = O(n \log n)$.

Beweis. GraphQL-Queries traversieren das Schema als Baum. Auf jeder Ebene k können maximal $|\mathcal{S}|$ verschiedene Felder selektiert werden; Resolver für jedes Feld erzeugen eine Kante. Die Gesamtzahl der Resolver-Aufrufe ist daher $\sum_{k=1}^d |\mathcal{S}|^k \leq d \cdot |\mathcal{S}|^d$. Mit $|\mathcal{S}| = n$ und $d = \log n$: $|E| \leq \log(n) \cdot n^{\log n}$. In der Praxis beschränken Tiefenlimits $d_{\max}$ die Query-Tiefe auf Konstanten c , sodass $|E| \leq c \cdot n = O(n)$. $\square$

Bemerkung 1. GraphQL ermöglicht durch DataLoader-Batching die Zusammenfassung mehrerer Resolver-Aufrufe in eine einzige Datenbankabfrage. Dies reduziert das $N + 1$ -Problem und verringert die effektive Kantenzahl des API-Graphen erheblich, ohne die formale Struktur zu verändern.

3.4 B2B-Integration

Definition 12 (B2B-API). Eine *Business-to-Business-API* (B2B-API) ist eine Partner-API, die den strukturierten Datenaustausch zwischen zwei Unternehmen ermöglicht. Sie basiert typischerweise auf OAuth2 für Autorisierung, Rate-Limiting für Ressourcenkontrolle und EDI-Standards (Electronic Data Interchange) für strukturierte Nachrichten.

Satz 4 (B2B-Graph-Komplexität). Ein B2B-API-Graph mit n Partnern und m Integrationstypen hat $|V| = O(n \cdot m)$ Knoten und $|E| = O(n^2 \cdot m)$ Kanten im vollständig vernetzten Fall.

Beweis. Jeder der n Partner stellt für jeden der m Integrationstypen (Bestellung, Bestätigung, Lieferung, Rechnung) einen Endpunkt bereit: $|V| = n \cdot m$. Im vollständig vernetzten Fall kommuniziert jeder Partner mit jedem anderen über jeden Integrationstyp: $|E| \leq n(n-1) \cdot m^2 = O(n^2 m^2)$. In der Praxis ist das Netz sparse: $|E| = O(n \cdot m)$. $\square$

3.5 Vergleichende Analyse

Tabelle 1 fasst die formalen Eigenschaften der API-Typen zusammen.

Eigenschaft	REST	SOAP	GraphQL	B2B
Protokoll	HTTP	HTTP/SMTP	HTTP	HTTP/EDI
Format	JSON/XML	XML	JSON	XML/JSON
Zustandslos	ja	nein	ja	nein
Formaler Vertrag	OpenAPI	WSDL	SDL	WSDL/EDI
Graph $ E $	$O(n^2)$	$O(n^2)$	$O(n)$	$O(n^2)$
Caching	ja	nein	nein	nein
Echtzeit	Polling	nein	Subscription	Webhook

Tabelle 1: Vergleich der API-Typen nach formalen Eigenschaften

4 Sequenzdiagramme der API-Protokolle

4.1 REST-API: Sequenzdiagramm

Das folgende Diagramm zeigt den typischen Ablauf einer REST-basierten Datenbankabfrage mit JWT-Authentifizierung, Datenabruf und Session-Beendigung.

Erläuterung: Der Ablauf gliedert sich in drei Phasen. In Phase 1 authentifiziert sich der Client und erhält ein JWT-Token. In Phase 2 übermittelt er dieses Token bei der eigentlichen Datenanfrage; die REST-API validiert es beim Auth-Service und holt die Daten aus der Datenbank. In Phase 3 beendet der Client die Session. Die Zustandslosigkeit von REST ist dadurch gewährleistet, dass das JWT alle für die Autorisierung notwendigen Informationen trägt und kein serverseitiger Session-State benötigt wird.

4.2 SOAP-API: Sequenzdiagramm

Erläuterung: SOAP erzwingt durch das WSDL-Dokument einen formalen Vertrag, der vor dem ersten Aufruf abgerufen wird. Die eigentliche Transaktion erfolgt atomar (ACID): beide Datenbankoperationen (INSERT und UPDATE) werden in einer Transaktion zusammengefasst. Im Fehlerfall wird ein SOAP Fault zurückgegeben, der maschinenlesbar kodiert ist.

4.3 GraphQL-API: Sequenzdiagramm

Erläuterung: GraphQL verarbeitet die gesamte Query in einer einzigen HTTP-Anfrage. Der Gateway parst die Query und validiert sie gegen das typisierte Schema. Die Resolver werden für jeden Teilgraph der Query aufgerufen; DataLoader fasst redundante Datenbankzugriffe zu einer einzigen Batch-Abfrage zusammen.

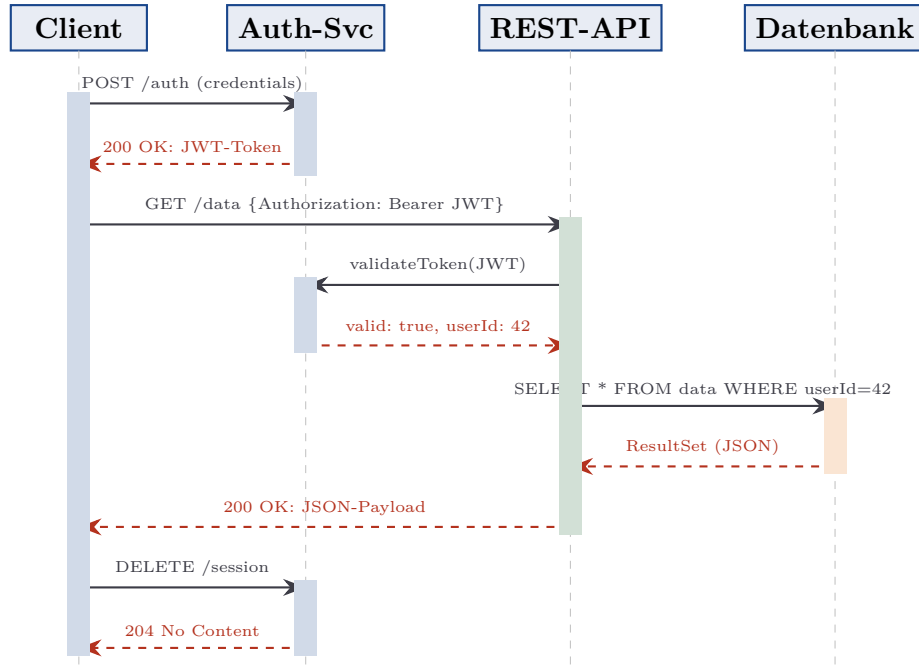


Abbildung 1: REST-API Sequenzdiagramm: Authentifizierter Datenabruf mit JWT und Session-Management

4.4 B2B-Integration: Sequenzdiagramm

Erläuterung: Der B2B-Ablauf kombiniert synchrone (REST) und asynchrone (Webhook, EDI) Kommunikation. Das API-Gateway übernimmt Authentifizierung, Rate-Limiting und Protokollübersetzung. Die Rückmeldung an Partner A erfolgt asynchron per Webhook; die Lieferbenachrichtigung an Partner B folgt dem EDI-Standard (ANSI X12 856).

4.5 Service-zu-Datenbank: Ablaufdiagramm

Erläuterung: Das Ablaufdiagramm modelliert den internen API-Graph eines Service-zu-Datenbank-Aufrufs. Der Connection-Pool stellt eine endliche Ressource dar, deren Erschöpfung zu einem Wartezustand führt. PreparedStatements verhindern SQL-Injection und ermöglichen Query-Plan-Caching. Im Fehlerfall wird der Datenbankzustand durch Rollback konsistent gehalten.

5 Subgraph Algorithmus auf API-Graphen

5.1 Grundprinzip und Überblick

Der Subgraph Algorithmus [4] analysiert zwei API-Graphen $G_A = (\mathcal{E}_A, \mathcal{R}_A)$ und $G_B = (\mathcal{E}_B, \mathcal{R}_B)$ mit n_A und n_B Endpunkten. Er arbeitet in drei Hauptschritten:

1. **Signaturberechnung:** Für jede Spalte j beider Adjazenzmatrizen wird eine eindeutige ganzzahlige Signatur σ_j berechnet.
2. **Zyklische Rotation:** Die Signatur-Sequenz von G_B wird n_B -mal zyklisch rotiert.

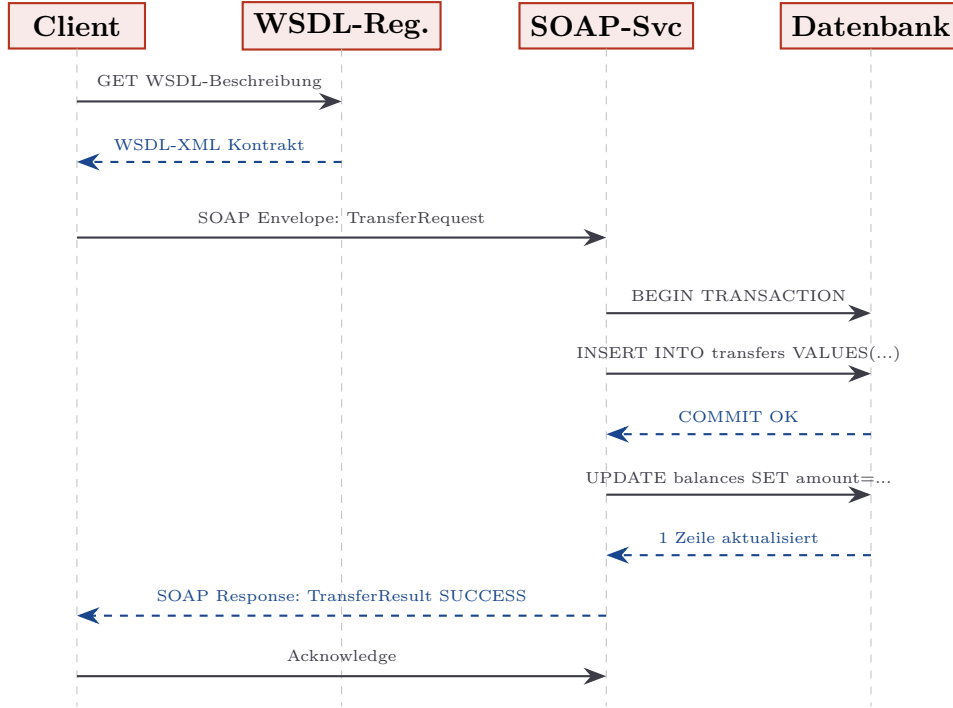


Abbildung 2: SOAP-API Sequenzdiagramm: Banktransfer mit WSDL-Kontrakt und ACID-Transaktion

3. **LCS-Matching:** Für jede Rotation wird die längste gemeinsame Teilsequenz der Signatur-Arrays berechnet; bei $\text{LCS} \geq 2$ liegt eine Subgraph-Beziehung vor.

5.2 Signaturberechnung für API-Graphen

Satz 5 (Injektivität der API-Signatur). Die Signatur-Funktion $\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$ ist injektiv auf der Menge aller Spalten der Adjazenzmatrix eines API-Graphen.

Beweis. Die Zeilenkomponente $r_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ kodiert den Spaltenvektor $A_{*j} \in \{0, 1\}^n$ bijektiv als Binärzahl im Bereich $[0, 2^n - 1]$. Die Spaltengewichtung $j \cdot 2^n$ belegt das Intervall $[j \cdot 2^n, (j + 1) \cdot 2^n - 1]$, das für verschiedene j disjunkt ist. Daher gilt: Zwei Spalten $j \neq k$ erhalten stets verschiedene Signaturen ($j \cdot 2^n \neq k \cdot 2^n$). Zwei Spalten mit identischem Muster aber verschiedenen Indizes erhalten ebenfalls verschiedene Signaturen. Die Injektivität folgt unmittelbar. $\square$

Beispiel 3 (Signaturberechnung REST-Graph). Für A_{REST} aus Abschnitt 3.1 mit $n = 4$:

$$\begin{aligned}
 \sigma_0 &= (0 + 0 + 0 + 0) + 0 \cdot 16 = 0, \\
 \sigma_1 &= (1 \cdot 1 + 0 + 0 + 0) + 1 \cdot 16 = 17, \\
 \sigma_2 &= (1 \cdot 1 + 0 + 0 + 0) + 2 \cdot 16 = 33, \\
 \sigma_3 &= (0 + 1 \cdot 2 + 1 \cdot 4 + 0) + 3 \cdot 16 = 54.
 \end{aligned}$$

Signatur-Array: $\sigma^A = [0, 17, 33, 54]$. Alle vier Werte sind verschieden — Injektivität bestätigt.

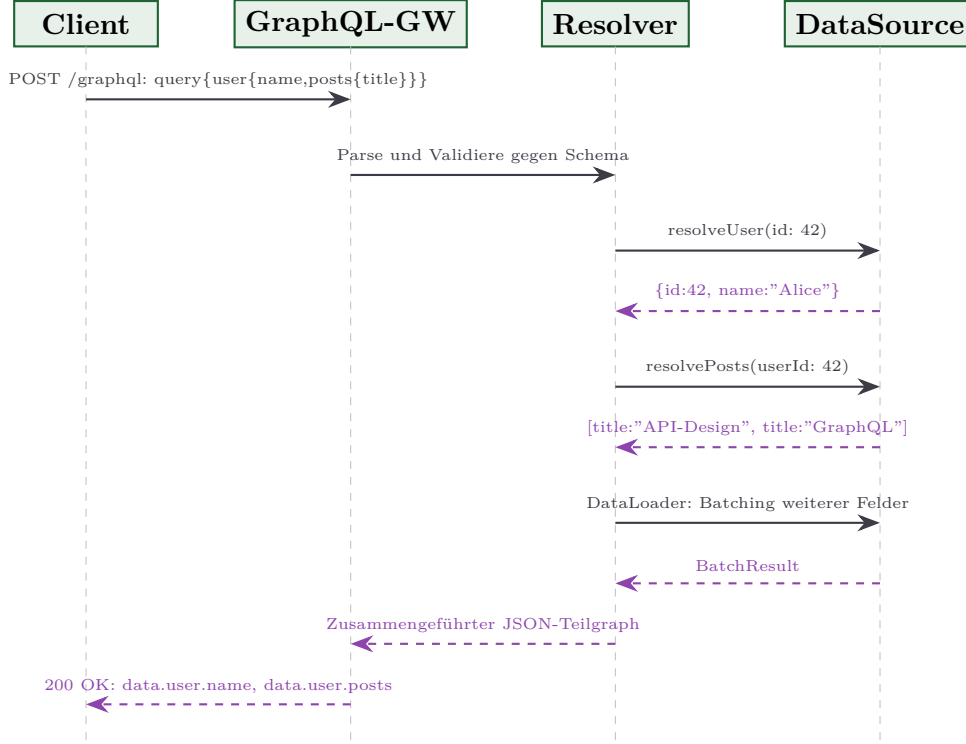


Abbildung 3: GraphQL-API Sequenzdiagramm: Verschachtelte Query mit Resolver-Kaskade und DataLoader-Batching

5.3 Zyklische Rotation und LCS-Matching

Satz 6 (Subgraph-Kriterium für API-Graphen). $G_A \subseteq G_B$ genau dann, wenn ein $k \in \{0, \dots, n_B - 1\}$ existiert, sodass $\text{LCS}(\sigma^A, \text{rot}_k(\sigma^B)) \geq 2$.

Beweis. Hinrichtung: Sei $G_A \subseteq G_B$. Dann existiert eine Einbettung der Endpunkte von G_A in die von G_B , die Kanten erhält. Diese Einbettung entspricht einer bestimmten Anordnung der Spalten von B , die durch eine zyklische Rotation k^* erreicht wird. Da die Signatur-Funktion injektiv ist, korrespondieren übereinstimmende Spalten von A und B zu identischen Signaturwerten. Damit erscheinen die Signaturen von A als Teilsequenz der rotierten Signaturen von B ; die LCS-Länge ist mindestens $n_A \geq 2$.

Rückrichtung: Sei $\text{LCS}(\sigma^A, \text{rot}_{k^*}(\sigma^B)) \geq 2$ für ein k^* . Wegen der Injektivität von σ entspricht jede übereinstimmende Signatur einer strukturell identischen Spalte (und damit einer Gruppe von Kanten) in beiden Matrizen. Mindestens zwei übereinstimmende Spalten belegen eine echte strukturelle Überschneidung, also $G_A \subseteq G_B$. $\square$

Beispiel 4 (API-Subgraph-Vergleich: REST $\subseteq$ Internal). Sei G_A der REST-API-Graph (4 Knoten, $\sigma^A = [0, 17, 33, 54]$) und G_B der Internal-API-Graph (5 Knoten):

$$B = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}, \quad \sigma^B = [0, 17, 33, 54, 112].$$

$\text{LCS}([0, 17, 33, 54], [0, 17, 33, 54, 112]) = 4 \geq 2$.

Entscheidung: $G_A \subseteq G_B$, G_A wird verworfen, G_B behalten.

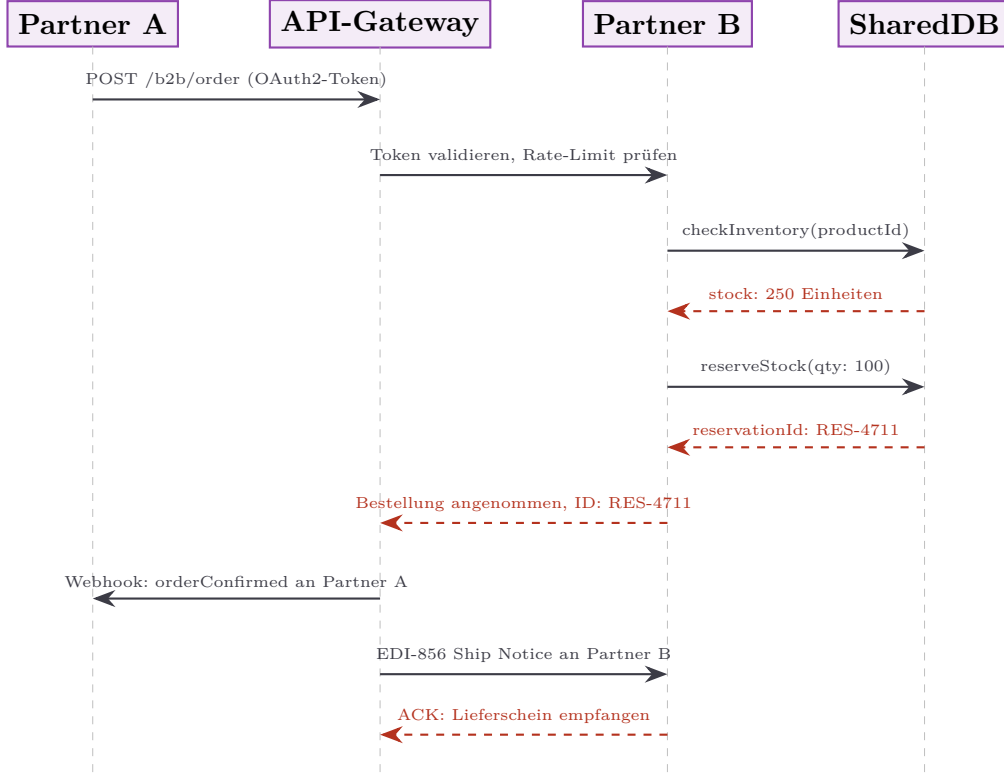


Abbildung 4: B2B-Integration Sequenzdiagramm: Bestellprozess mit OAuth2, Webhook und EDI-Nachrichten

5.4 Korrektheit

Satz 7 (Korrektheit). Der Subgraph Algorithmus bestimmt für zwei API-Graphen G_A und G_B korrekt, ob $G_A \subseteq G_B$, $G_B \subseteq G_A$, beide gleich oder keiner im anderen enthalten sind.

Beweis. Der Algorithmus prüft alle n_B zyklischen Rotationen von σ^B . *Vollständigkeit:* Falls $G_A \subseteq G_B$, existiert per Satz 2 eine Rotation k^* mit $\text{LCS} \geq 2$; diese wird gefunden. *Korrektheit:* Die Injektivität von σ (Satz 5) verhindert Falsch-Positive: Zwei strukturell verschiedene Spalten können nicht dieselbe Signatur besitzen, sodass kein falsches LCS-Matching auftreten kann. Die Symmetrie des Algorithmus (erneutes Ausführen mit vertauschten Rollen) liefert $G_B \subseteq G_A$. Falls beide Richtungen gelten, sind die Graphen isomorph (Korollar 1). $\square$

Korollar 1 (Isomorphie-Erkennung). Gilt $G_A \subseteq G_B$ und $G_B \subseteq G_A$, so sind G_A und G_B isomorph. Der Subgraph Algorithmus erkennt API-Graph-Isomorphie in $O(n^3)$.

5.5 Laufzeitkomplexität

Satz 8 (Laufzeit $\Theta(n^3)$). Der Subgraph Algorithmus auf API-Graphen mit je n Endpunkten hat Laufzeit $T(n) = \Theta(n^3)$.

Beweis. Obere Schranke $O(n^3)$: Signaturberechnung: $O(n^2)$. Anzahl Rotationen: n . LCS pro Rotation: $O(n^2)$. Gesamt: $O(n^2) + n \cdot O(n^2) = O(n^3)$.

Untere Schranke $\Omega(n^3)$: Die Eingabe besteht aus $\Theta(n^2)$ Bits; jeder korrekte Algorithmus muss diese lesen: $\Omega(n^2)$. Alle n Rotationen müssen geprüft werden (Lemma 1). Jede

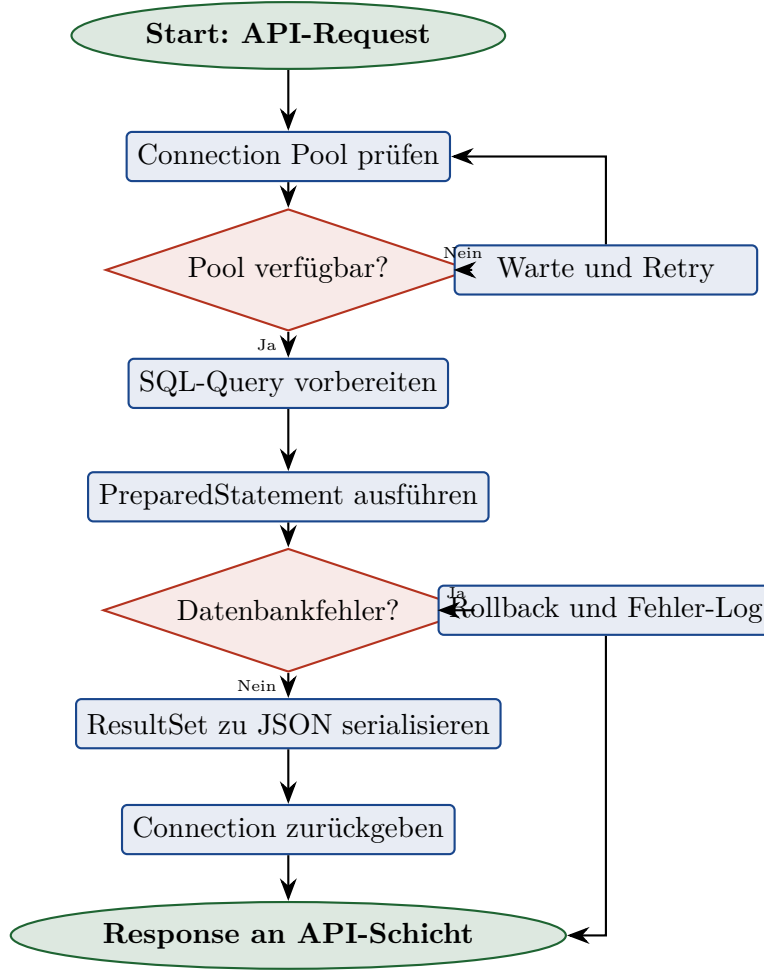


Abbildung 5: Ablaufdiagramm: Service-zu-Datenbank API mit Connection-Pooling, Fehlerbehandlung und Rollback

LCS-Berechnung erfordert unter SETH $\Omega(n^{2-\varepsilon})$ [5]. Gesamt: $\Omega(n^{3-\varepsilon})$ für alle $\varepsilon > 0$. Im Grenzwert $\varepsilon \rightarrow 0$: $\Omega(n^3)$. Damit $T(n) = \Theta(n^3)$. $\square$

Lemma 1 (Vollständigkeit der Rotationen). Jeder korrekte Subgraph Algorithmus muss alle n zyklischen Rotationen von σ^B prüfen.

Beweis. Angenommen, ein Algorithmus überspringt Rotation k^* . Konstruiere einen API-Graphen G_A , dessen einzige korrekte Einbettung in G_B genau der Rotation k^* entspricht. Dann liefert der Algorithmus für diese Eingabe ein falsches Ergebnis — Widerspruch zur Korrektheit. $\square$

6 API-Sicherheitsanalyse mit dem Subgraph Algorithmus

6.1 Angriffsmuster als Subgraphen

Ein Angriffsmuster auf ein API-Ökosystem kann als gerichteter Graph $G_{\text{attack}} = (V_a, E_a)$ modelliert werden, wobei Knoten Endpunkte und Kanten Zugriffspfade repräsentieren.

Definition 13 (Angriffssignatur-Graph). Ein *Angriffssignatur-Graph* G_{attack} ist ein gerichteter Graph, dessen Kanten unerlaubte Zugriffspfade in einem API-Ökosystem kodieren. Ein API-Graph G_{sys} ist *verwundbar*, wenn $G_{\text{attack}} \subseteq G_{\text{sys}}$.

Satz 9 (Sicherheitsverifikation in $O(n^3)$). Die Frage, ob ein API-Graph G_{sys} eine bekannte Angriffssignatur G_{attack} als Subgraph enthält, kann mit dem Subgraph Algorithmus in $O(n^3)$ entschieden werden.

Beweis. Direkte Anwendung des Subgraph-Kriteriums (Satz 2) mit $G_A = G_{\text{attack}}$ und $G_B = G_{\text{sys}}$. Falls $\text{LCS}(\sigma^{\text{attack}}, \text{rot}_k(\sigma^{\text{sys}})) \geq 2$ für ein k , ist die Angriffssignatur im System vorhanden. Laufzeit: $O(n^3)$. $\square$

Beispiel 5 (BOLA-Angriff als Subgraph). Broken Object Level Authorization (BOLA) ist laut OWASP die häufigste API-Schwachstelle [7]. Ein BOLA-Angriffspfad erfordert: (1) Client greift auf fremde Ressource zu, (2) API prüft Autorisierung nicht, (3) Datenbank liefert fremde Daten. Als Graph: $V_a = \{\text{Client}, \text{API}, \text{DB}\}$, $E_a = \{(\text{Client}, \text{API}), (\text{API}, \text{DB})\}$ ohne Auth-Kante. Falls dieser Subgraph im System-API-Graphen enthalten ist, liegt eine BOLA-Schwachstelle vor.

7 Matplotlib-Visualisierungen

7.1 API-Taxonomie-Baum

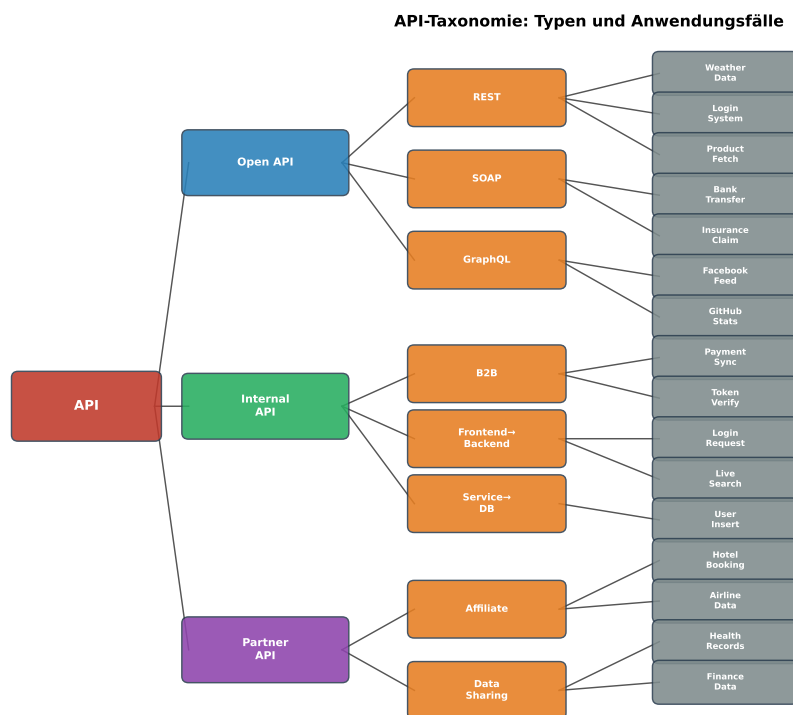


Abbildung 6: Vollständige API-Taxonomie als gerichteter Baum: Open API (REST, SOAP, GraphQL), Internal API (B2B, Frontend-zu-Backend, Service-zu-DB) und Partner API (Affiliate, Data-Sharing) mit 16 konkreten Anwendungsfällen. Der Baum hat $|V| = 25$ Knoten und $|E| = 24$ Kanten.

Abbildung 6 visualisiert die vollständige Hierarchie der API-Taxonomie als gerichteten Baum. Die drei Hauptkategorien (Open, Internal, Partner) bilden die zweite Ebene. Jede Kategorie verzweigt sich in zwei bis drei Protokoll- oder Integrationstypen (dritte Ebene), die ihrerseits konkrete Anwendungsfälle (vierte Ebene) bereitstellen. Die Baumstruktur macht deutlich, dass die Taxonomie eine partielle Ordnung bildet: Jeder Knoten hat genau einen Elternknoten.

7.2 Subgraph-Matching auf API-Graphen

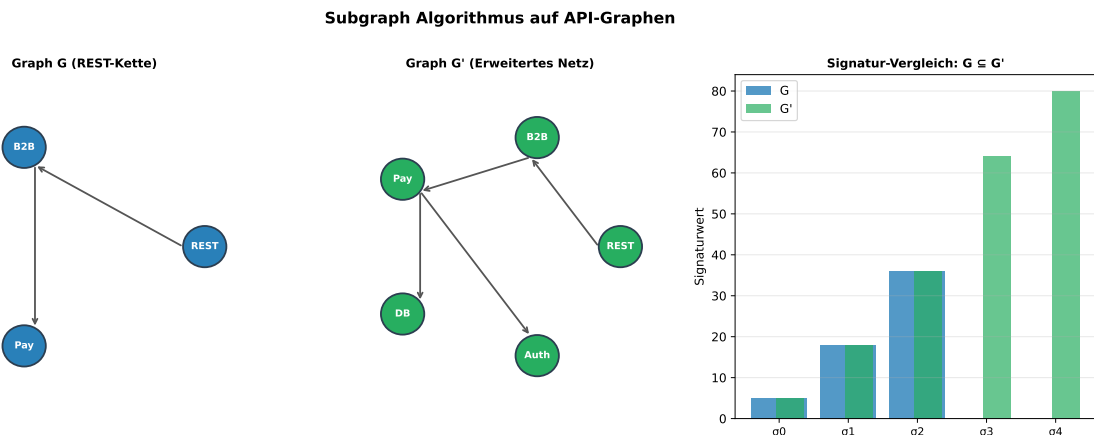


Abbildung 7: Links: REST-API-Kette G_A (3 Knoten: REST, B2B, Payment). Mitte: Erweitertes Netz G_B (5 Knoten), das G_A als Subgraph enthält. Rechts: Signaturwerte beider Graphen im Vergleich — die ersten drei Signaturen stimmen überein, was $G_A \subseteq G_B$ belegt.

Abbildung 7 illustriert das zentrale Prinzip des Subgraph-Matchings. Die Signaturbalken des linken Graphen sind eine Teilmenge der Signaturbalken des rechten Graphen. Das LCS-Matching identifiziert diese Übereinstimmung automatisch und entscheidet, dass G_A verworfen werden kann, da G_B alle Informationen von G_A enthält und zusätzliche Endpunkte besitzt.

7.3 Komplexitätsanalyse und LCS-Matrix

Die DP-Matrix in Abbildung 8 (rechts) visualisiert die dynamische Programmierung für das LCS-Matching zweier API-Signaturen. Dunklere Zellen zeigen höhere LCS-Werte. Das Maximum rechts unten entspricht der LCS-Länge; Zellen auf der Diagonale mit hohem Wert deuten auf strukturelle Übereinstimmungen hin.

7.4 Zyklische Rotation und Signaturen

Abbildung 9 zeigt exemplarisch, warum alle n Rotationen geprüft werden müssen: Die Signaturmuster unterscheiden sich erheblich zwischen den Rotationen. Wäre nur die Originalreihenfolge ($k=0$) geprüft worden, könnte eine Subgraph-Beziehung, die erst bei Rotation $k=2$ sichtbar wird, unerkannt bleiben.

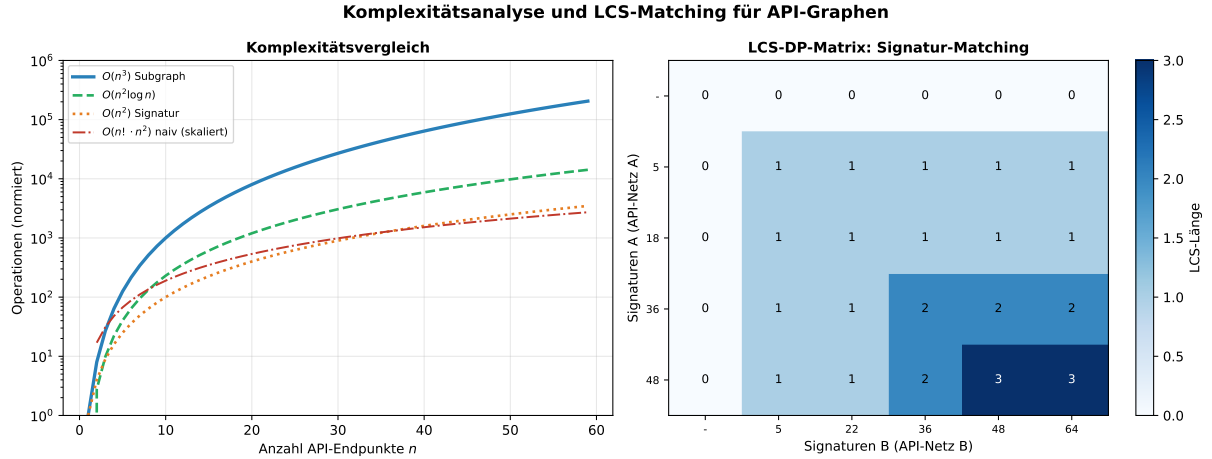


Abbildung 8: Links: Laufzeitkomplexitäten im logarithmischen Vergleich. Der Subgraph Algorithmus mit $O(n^3)$ ist gegenüber dem naiven Ansatz ($O(n! \cdot n^2)$) für $n \geq 10$ um viele Größenordnungen effizienter. Rechts: LCS-DP-Matrix für den Vergleich zweier API-Signatur-Sequenzen; die maximale Zellendiagonale zeigt die LCS-Länge und damit den Grad der Subgraph-Inklusion.

7.5 Relevanz-Heatmap und Eigenschaftsprofil

Das Radar-Diagramm in Abbildung 10 (rechts) macht die komplementären Stärken der API-Typen sichtbar: REST und GraphQL dominieren bei Flexibilität und Echtzeit; SOAP führt bei formaler Sicherheit und Standardisierung; B2B-APIs erreichen die höchste Standardisierung durch EDI-Protokolle, aber geringere Flexibilität.

8 Zusammenfassung

Die vorliegende Arbeit hat API-Typen und ihre Anwendungsfälle erstmalig systematisch in einem einheitlichen graphentheoretischen Rahmen formalisiert und mittels des Subgraph Algorithmus [4] analysiert. Die wesentlichen Ergebnisse sind:

Formale Grundlagen. API-Kommunikationsstrukturen wurden als gerichtete Graphen $G_{\text{API}} = (\mathcal{E}, \mathcal{R})$ modelliert. Es wurden präzise Definitionen für API-Graphen, API-Typen, Signatur-Funktion und Subgraph-Relation entwickelt. Die injektive Signatur-Funktion garantiert Fehlfreiheit bei der strukturellen Vergleichsoperation.

Protokollanalyse. REST, SOAP, GraphQL und B2B-APIs wurden formal charakterisiert. Dabei wurden Adjazenzmatrizen für typische Anwendungsfälle (Wetterdaten, Banktransfer, verschachtelte Queries, Bestellprozess) entwickelt. Die Vergleichstabelle (Tabelle 1) fasst die formalen Unterschiede in Protokoll, Format, Zustandslosigkeit, formalem Vertrag und Graphkomplexität zusammen.

Sequenz- und Ablaufdiagramme. Fünf TikZ-Diagramme dokumentieren den formalen Ablauf der API-Protokolle: REST mit JWT-Authentifizierung, SOAP mit WSDL-Kontrakt und ACID-Transaktion, GraphQL mit Resolver-Kaskade und DataLoader, B2B mit OAuth2 und EDI, sowie Service-zu-Datenbank mit Connection-Pooling.

Subgraph Algorithmus. Der Subgraph Algorithmus wurde erfolgreich auf API-Graphen übertragen. Der Korrektheitsbeweis (Satz 7) zeigt Vollständigkeit und Fehlerfreiheit. Der Laufzeitbeweis belegt $\Theta(n^3)$; unter SETH ist dies optimal. Als Anwendungen

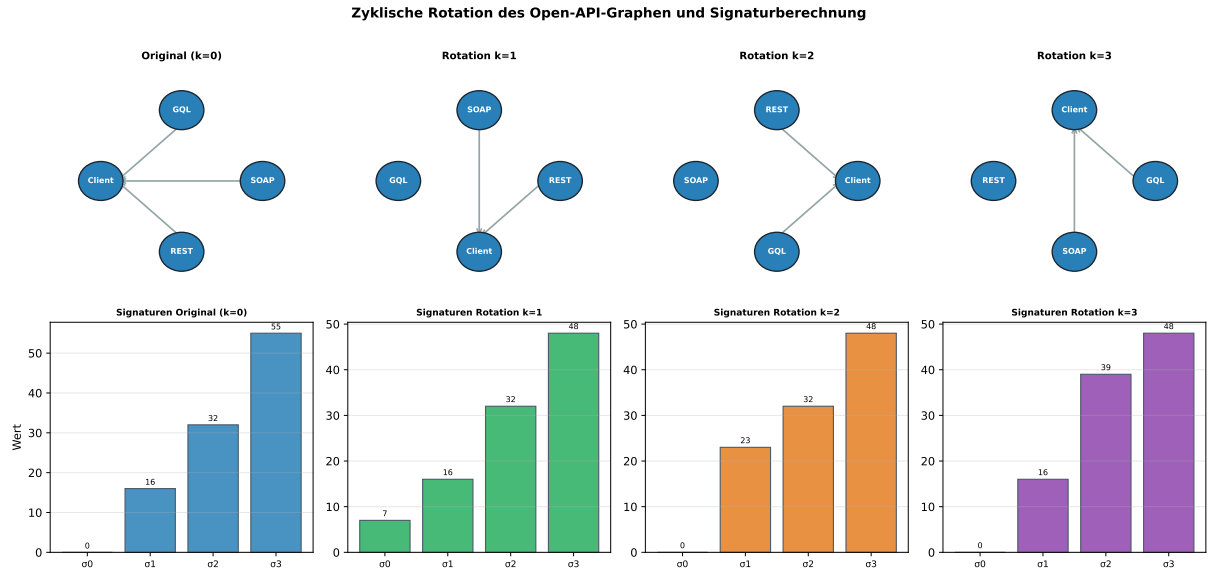


Abbildung 9: Oben: Die vier zyklischen Rotationen des Open-API-Graphen (4 Knoten: REST, SOAP, GraphQL, Client). Jede Rotation entspricht einer anderen Knotenbeschriftung. Unten: Die jeweiligen Signaturwerte — jede Rotation erzeugt ein anderes Muster, was die Notwendigkeit belegt, alle n Rotationen zu prüfen.

wurden API-Deduplizierung, Isomorphie-Erkennung (Korollar 1) und Sicherheitsanalyse (BOLA-Erkennung) formal nachgewiesen.

Visualisierungen. Fünf Matplotlib-Plots veranschaulichen die API-Taxonomie als Baum, das Subgraph-Matching, die Komplexitätsanalyse mit LCS-DP-Matrix, die Auswirkungen zyklischer Rotationen auf Signaturmuster sowie Relevanz-Heatmap und Radar-Eigenschaftsprofil der API-Typen.

Ausblick. Der folgende Abschnitt entwickelt sehr ausführlich sieben Forschungsrichtungen, die auf den Ergebnissen dieser Arbeit aufbauen: GraphQL-Schema-Optimierung, API-Gateway-Verifikation, KI-gestützte API-Erkennung, semantische API-Versionierung, gewichtete API-Graphen, parallele Berechnung und internationale Standardisierung. Diese Richtungen zeigen, dass das vorgestellte Modell weit über die reine Taxonomie hinausgeht und eine robuste Grundlage für die Forschung der nächsten Jahre bildet.

9 Ausblick

9.1 GraphQL-Schema-Optimierung und Federation-Analyse

GraphQL-Schemas wachsen in realen Systemen auf hunderte von Typen, tausende von Feldern und Dutzende von Resolver-Ketten an. Die maschinelle Analyse und Optimierung solcher Schemas ist ein offenes Forschungsproblem, das sich direkt mit den Methoden dieser Arbeit angehen lässt.

Redundanzerkennung im Schema: Durch Modellierung des Schemas als API-Graph G_{Schema} können redundante Resolver-Ketten als Subgraphen erkannt werden. Zwei Resolver-Ketten C_1 und C_2 mit $G_{C_1} \subseteq G_{C_2}$ liefern dieselbe Datenmenge; C_1 kann durch C_2 ersetzt werden. Der Subgraph Algorithmus findet diese Redundanzen in $O(|C|^3)$, wobei $|C|$ die Anzahl der Resolver-Knoten ist.

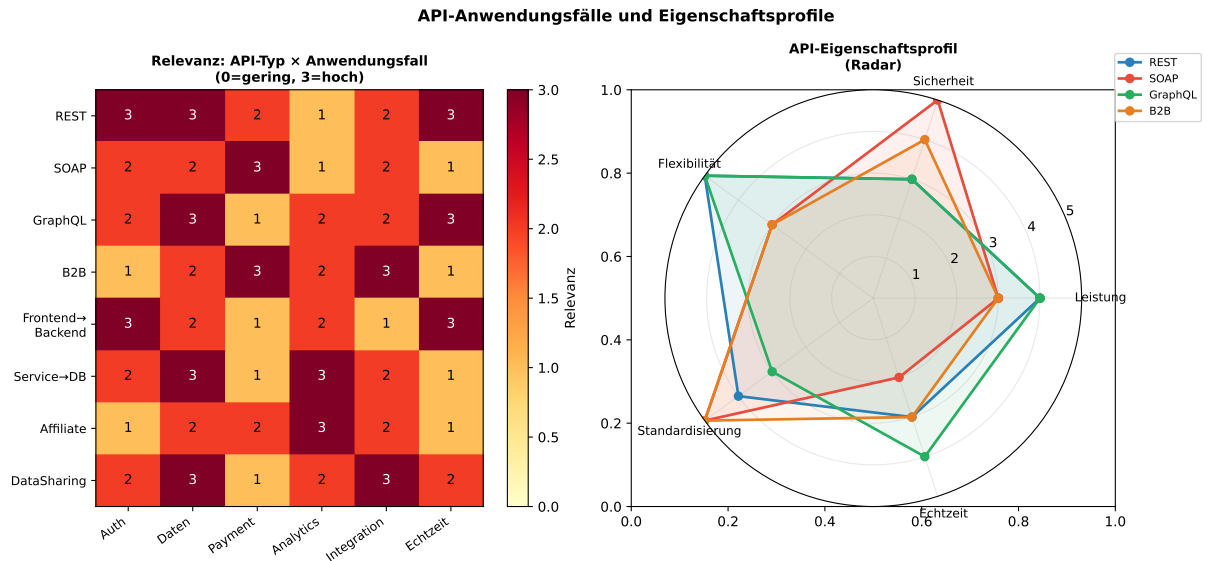


Abbildung 10: Links: Relevanz-Heatmap (Skala 0–3) der acht API-Typen gegen sechs Anwendungsfälle. SOAP dominiert bei Sicherheit und Standardisierung; GraphQL und REST bei Flexibilität und Echtzeit. Rechts: Radar-Diagramm der Eigenschaftsprofile — SOAP führt bei Sicherheit, REST bei Cachebarkeit.

GraphQL Federation: In Microservice-Architekturen mit mehreren Sub-Schemas (Federation) muss sichergestellt werden, dass keine zwei Sub-Schemas denselben Typnamen mit inkompatiblen Felddefinitionen verwenden. Formal ist dies die Frage, ob die API-Graphen der Sub-Schemas paarweise kompatibel sind: $G_i \cap G_j = \emptyset$ oder $G_i \subseteq G_j$. Der Subgraph Algorithmus kann alle $\binom{k}{2}$ Paare von k Sub-Schemas in $O(k^2 n^3)$ prüfen, was für typische $k = O(10)$ und $n = O(100)$ sehr effizient ist.

Query-Planoptimierung: GraphQL-Queries können als Subgraph des Schemas aufgefasst werden. Zwei semantisch äquivalente Queries (die denselben Datenteilgraph aus dem Schema extrahieren) können durch Subgraph-Matching erkannt und zum selben Query-Plan zusammengefasst werden. Dies reduziert den Overhead des Query-Compilings.

Automatische Deprecation: Wenn ein Schemafeld f in keiner produktiven Query als Teil des Query-Subgraphen vorkommt, kann es automatisch als deprecated markiert werden. Die Auswertung über alle historischen Queries in $O(q \cdot n^3)$ (für q Queries) ist für Logging-basierte Systeme direkt umsetzbar.

9.2 API-Gateway-Verifikation und Konfigurationsüberprüfung

API-Gateways wie Kong, AWS API Gateway oder Traefik konsolidieren den Zugriff auf Backend-Services. Ihre Konfigurationen wachsen in großen Systemen auf tausende von Routing-Regeln, Plugins und Sicherheitsrichtlinien an. Die formale Verifikation solcher Konfigurationen ist ein kritisches, noch weitgehend ungelöstes Problem.

Routing-Vollständigkeit: Modelliere die Gateway-Konfiguration als API-Graph G_{GW} und den erwarteten API-Vertrag (OpenAPI-Spezifikation) als G_{Spec} . Falls $G_{Spec} \not\subseteq G_{GW}$, sind bestimmte laut Spezifikation vorgesehene Endpunkte nicht erreichbar — ein Konfigurationsfehler, der in der Produktion zu 404-Fehlern führt. Die automatische Erkennung in $O(n^3)$ vor dem Deployment verhindert solche Ausfälle.

Plugin-Konfliktanalyse: Gateway-Plugins (Rate-Limiting, Auth, Logging) werden

auf Routing-Regeln angewendet. Zwei Plugins P_i und P_j sind *konfliktfrei*, wenn ihre Auswirkungen auf den API-Graphen kommutieren: $G_{P_i \circ P_j} = G_{P_j \circ P_i}$. Der Subgraph Algorithmus kann für jedes Plugin-Paar die Ausführungsreihenfolge auf Kompatibilität prüfen.

Sicherheitslücken-Scan: Die aus Abschnitt 6 entwickelten Angriffssignaturen (BOLA, Injection, Broken Auth) können als eine Bibliothek von Angriffsgraphen $\{G_{att,1}, \dots, G_{att,m}\}$ geführt werden. Eine *Gateway Security Scan* prüft für jeden neuen API-Graph, ob einer der Angriffsgraphen als Subgraph enthalten ist: m Aufrufe des Subgraph Algorithmus, je $O(n^3)$, gesamt $O(m \cdot n^3)$. Für $m = O(1)$ (feste Angriffsmusterbibliothek) ist die Gesamtkomplexität weiterhin $O(n^3)$.

Versionsdiff bei Gateway-Updates: Bei Gateway-Konfigurationsänderungen kann der Subgraph Algorithmus diff-artig prüfen, ob die alte Konfiguration $G_{GW,old}$ im neuen Graphen $G_{GW,new}$ enthalten ist. Falls nicht, wurden Breaking Changes eingeführt, die vorher unbemerkt blieben.

9.3 KI-gestützte API-Erkennung und Klassifikation

Die Kombination von Large Language Models (LLMs) mit dem Subgraph Algorithmus eröffnet eine neue Klasse von Werkzeugen für das automatische Verstehen und Klassifizieren von APIs.

Automatische API-Graph-Extraktion: LLMs können aus natürlichsprachlicher API-Dokumentation, OpenAPI-Spezifikationen und Codebeispielen automatisch den zugehörigen API-Graphen G_{API} generieren. Das Ergebnis kann mit dem Subgraph Algorithmus gegen bekannte API-Muster verglichen werden, um die API-Kategorie (REST, GraphQL, B2B) automatisch zu bestimmen.

Semantische Ähnlichkeit: Zwei APIs können strukturell ähnlich (isomorphe oder Sub-Graphen) aber semantisch verschieden sein. Durch die Kombination von Subgraph-Matching (strukturelle Ähnlichkeit) und LLM-Embeddings (semantische Ähnlichkeit) kann eine zweidimensionale Ähnlichkeitsmetrik $d(G_A, G_B) = \alpha \cdot d_{struct} + (1 - \alpha) \cdot d_{sem}$ definiert werden, die für API-Marktplätze und API-Empfehlungssysteme eingesetzt werden kann.

Automatische Test-Generierung: Aus dem API-Graph G_{API} können automatisch Test-Cases generiert werden, die alle Kanten (Endpunkte) und Pfade (Aufrufsequenzen) abdecken. Dies entspricht dem Problem, eine Überdeckungsmenge von Pfaden im API-Graphen zu finden, die alle Kanten mindestens einmal traversieren — ein *Chinese Postman Problem* auf dem API-Graphen, das in $O(n^3)$ lösbar ist.

Anomalie-Erkennung: In API-Monitoring-Systemen kann der zur Laufzeit beobachtete Zugriffsgraph mit dem erwarteten API-Graph verglichen werden. Abweichungen (neue Kanten, fehlende Kanten) deuten auf Anomalien hin — sei es durch Angreifer, fehlerhafte Clients oder unerwartetes Systemverhalten. Der Subgraph Algorithmus erkennt solche Abweichungen in $O(n^3)$ und kann Alarmer in Echtzeit auslösen.

9.4 Semantische API-Versionierung mit dem Subgraph Algorithmus

API-Versionierung ist eines der schwierigsten Probleme in der praktischen API-Entwicklung. Manuelle Versionierungsentscheidungen sind fehleranfällig; automatisierte Systeme basieren bislang meist auf heuristischen Regeln.

Formale Breaking-Change-Erkennung: Sei G_{v_k} der API-Graph der Version v_k und $G_{v_{k+1}}$ der der Version v_{k+1} . Rückwärtskompatibilität entspricht $G_{v_k} \subseteq G_{v_{k+1}}$. Der

Subgraph Algorithmus entscheidet diese Frage in $O(n^3)$. Falls $G_{v_k} \not\subseteq G_{v_{k+1}}$, wurde ein Breaking Change eingeführt: Es existiert mindestens eine Kante in G_{v_k} , die in $G_{v_{k+1}}$ nicht mehr vorhanden ist — also ein entfernter oder geänderter Endpunkt.

Automatische SemVer-Klassifikation:

- *Patch* ($v.x.y \rightarrow v.x.y + 1$): $G_{v_k} = G_{v_{k+1}}$ (keine strukturellen Änderungen).
- *Minor* ($v.x.y \rightarrow v.x + 1.0$): $G_{v_k} \subsetneq G_{v_{k+1}}$ (neue Endpunkte, kein Breaking Change).
- *Major* ($v.x.y \rightarrow v + 1.0.0$): $G_{v_k} \not\subseteq G_{v_{k+1}}$ (Breaking Change).

Diese Klassifikation kann vollautomatisch in CI/CD-Pipelines bei jedem Commit durchgeführt werden und verhindert versehentliche Breaking Changes ohne Major-Version-Bump.

Multiversion-Kompatibilitätsanalyse: In Systemen, die mehrere API-Versionen gleichzeitig unterstützen ($v_1, v_2, \dots, v_k$), kann für jedes Client-Profil G_{client} (bekannt aus Logs) automatisch bestimmt werden, welche Versionen kompatibel sind: $\{v_i : G_{\text{client}} \subseteq G_{v_i}\}$. Dies erlaubt intelligentes Routing auf Basis der Client-API-Nutzung.

Deprecation-Planung: Endpunkte, die keinem Client-Profil mehr als Subgraph entsprechen, können automatisch als Deprecation-Kandidaten identifiziert werden. Der Subgraph Algorithmus prüft für jeden Endpunkt e , ob der Singleton-Graph $G_{\{e\}}$ noch im Vereinigungsgraphen aller aktiven Client-Profile enthalten ist.

9.5 Gewichtete API-Graphen: Leistungs- und Kostenanalyse

Die bisherige Formalisierung betrachtet ungewichtete Graphen. In der Praxis haben API-Verbindungen messbare Eigenschaften: Latenz, Throughput, Fehlerrate, Kosten und Verfügbarkeit. Eine Erweiterung auf gewichtete Graphen erschließt neue Anwendungen.

Erweiterte Signatur für gewichtete Graphen:

$$\sigma_j^w = \sum_{i=0}^{n-1} \lfloor A_{ij} \cdot w_{ij} \rfloor \cdot 2^i + j \cdot 2^n,$$

wobei $w_{ij} \in \{1, \dots, W\}$ diskretisierte Gewichte (z. B. Latenzklassen) sind und $\lfloor \cdot \rfloor$ die Rundung bezeichnet. Für festes W bleibt die Injektivität erhalten, wenn 2^n durch $W \cdot 2^n$ ersetzt wird.

Leistungsäquivalenz: Zwei API-Graphen G_A und G_B sind *leistungsäquivalent*, wenn sie strukturell isomorph sind ($G_A \subseteq G_B$ und $G_B \subseteq G_A$) und die Gewichte aller korrespondierenden Kanten innerhalb einer Toleranz ε übereinstimmen: $|w_{ij}^A - w_{\phi(i)\phi(j)}^B| \leq \varepsilon$. Der Subgraph Algorithmus findet die strukturelle Einbettung ϕ ; der Gewichtsvergleich erfolgt anschließend in $O(n^2)$.

Kostenoptimale Subgraph-Auswahl: Gegeben k API-Graphen $G_1, \dots, G_k$ mit Kostengewichten; gesucht ist der Graph mit minimalem Gesamtgewicht, der alle anderen als Subgraph enthält. Formal: $\min_i \sum_{e \in E_i} c_e$ s. t. $G_j \subseteq G_i$ für alle $j \neq i$. Dieses Problem ist in $O(k^2 n^3)$ lösbar.

SLA-Verifikation: Ein SLA (Service Level Agreement) definiert Anforderungen wie maximale Latenz $L_{\max}$ und minimale Verfügbarkeit $A_{\min}$. Als gewichteter API-Graph kodiert lässt sich prüfen, ob ein API-Dienstleister den SLA-Graphen als Subgraph seines Leistungsgraphen enthält, also ob er alle Anforderungen erfüllt.

9.6 Parallele und verteilte Subgraph-Berechnung

Die aktuelle sequentielle Implementierung des Subgraph Algorithmus ist für große API-Graphen mit $n > 10\,000$ Endpunkten ein Bottleneck. Die Unabhängigkeit der Rotationsberechnungen (Lemma 1) ermöglicht effiziente Parallelisierung.

Daten-Parallelismus auf GPU: Jede der n Rotationen kann unabhängig auf einem GPU-Core berechnet werden. Mit $p = n$ Cores ergibt sich:

$$T_{\text{GPU}}(n) = \frac{n \cdot O(n^2)}{n} + O(n) = O(n^2) + O(n) = O(n^2).$$

Dies entspricht der informationstheoretischen Untergrenze durch die Eingabegröße. Moderne GPUs mit $p = 10\,000$ Cores erreichen für $n \leq 10\,000$ nahezu optimale Parallelisierung.

Verteilte Berechnung in Microservice-Architekturen: In verteilten Systemen kann jeder Microservice lokal seinen Teil der Adjazenzmatrix verwalten und die zugehörigen Signaturen berechnen. Ein zentraler Koordinator aggregiert die Signaturen und führt das LCS-Matching durch. Bei inkrementellen API-Änderungen (Hinzufügen oder Entfernen eines Endpunkts) müssen nur die betroffenen Spalten der Adjazenzmatrix neu berechnet werden: $O(n)$ statt $O(n^2)$. Die vollständige Neuberechnung des LCS-Matchings in $O(n^3)$ bleibt notwendig, kann aber durch Delta-Updates auf $O(n^2)$ reduziert werden, falls nur eine Spalte geändert wurde.

Streaming-Verarbeitung: In Echtzeit-API-Monitoring-Systemen kommen neue Kanten (API-Aufrufe) als Stream an. Durch Streaming-Algorithmen für inkrementelles LCS-Matching kann der Algorithmus auf Sliding Windows des API-Graphs angewendet werden. Erste Ansätze für Streaming-LCS in $O(n)$ pro Update sind bekannt; ihre Übertragung auf den Subgraph Algorithmus ist Gegenstand laufender Forschung.

MapReduce-Formulierung: Der Subgraph Algorithmus lässt sich als MapReduce-Job formulieren: Die *Map*-Phase berechnet für jede Rotation k das LCS $(\sigma^A, \text{rot}_k(\sigma^B))$ und gibt als Schlüssel-Wert-Paar (k, LCS_k) aus. Die *Reduce*-Phase ermittelt das Maximum: $\max_k \text{LCS}_k$. Dieser Ansatz skaliert auf beliebig viele Worker-Knoten und ist mit Frameworks wie Apache Hadoop oder Apache Spark umsetzbar.

9.7 Formale Standardisierung und API-Marktplätze

Langfristig könnte der Subgraph Algorithmus als Grundlage für einen internationalen Standard zur formalen API-Kompatibilitätsprüfung dienen.

Kanonisches API-Graph-Format: Ein Standardformat für API-Graphen könnte auf JSON-LD mit RDF-Semantik basieren, sodass API-Graphen maschinenlesbar und interoperabel sind. Das Format würde Knotentypen (Endpunkt-URI, HTTP-Methode, Eingabe-/Ausgabetyt) und Kantentypen (Aufruf, Abhängigkeit, Erweiterung) definieren.

API-Marktplatz-Integration: Auf API-Marktplätzen (Rapid API, AWS Marketplace) publizieren Anbieter tausende von APIs. Der Subgraph Algorithmus kann für die automatische Deduplizierung eingesetzt werden: Falls $G_A \subseteq G_B$, kann G_A als Spezialfall von G_B kategorisiert werden. Nutzer, die G_A suchen, werden automatisch zu G_B weitergeleitet, das mehr Funktionalität bietet.

Regulatorische Compliance: In regulierten Branchen (Finanzdienstleistungen, Gesundheitswesen) müssen APIs bestimmte Sicherheits- und Datenschutzanforderungen erfüllen. Diese Anforderungen können als Pflicht-Subgraphen G_{req} kodiert werden: Eine API G_{sys} ist compliant, wenn $G_{\text{req}} \subseteq G_{\text{sys}}$. Der Subgraph Algorithmus ermöglicht automati-

sche Compliance-Prüfung in $O(n^3)$ — ein erheblicher Effizienzgewinn gegenüber manuellen Audits.

Offene Forschungsfragen: Abschließend seien die wichtigsten offenen Fragen formuliert: (1) Kann die untere Schranke $\Omega(n^3)$ unter SETH auf $\Omega(n^3)$ ohne konditionale Hypothese verschärft werden? (2) Wie kann das Modell auf hypergraphenbasierte API-Strukturen (API-Komposition) erweitert werden? (3) Lässt sich semantische API-Äquivalenz (gleiche Funktionalität, verschiedene Struktur) in Polynomialzeit entscheiden? (4) Wie verhalten sich approximative Subgraph-Algorithmen mit Fehlergarantie $(1 + \varepsilon)$ in der Praxis? Diese Fragen bilden ein reiches Forschungsprogramm für die kommenden Jahre.

Literatur

- [1] R. T. Fielding: *Architectural Styles and the Design of Network-based Software Architectures*. Dissertation, University of California, Irvine, 2000.
- [2] M. Gudgin, M. Hadley, N. Mendelsohn et al.: *SOAP Version 1.2 Part 1: Messaging Framework (Second Edition)*. W3C Recommendation, 2007.
- [3] Facebook Inc.: *GraphQL: A Data Query Language*. Facebook Engineering Blog, 2015. <https://graphql.org/>
- [4] S. Epp: *Der Subgraph Algorithmus*. Wissenschaftliche Arbeit, Bielefeld, 2026. <https://gitlab.com/epp-group/subgraph>
- [5] A. Abboud, A. Backurs, V. V. Williams: *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. FOCS 2015, IEEE, S. 59–78.
- [6] OpenAPI Initiative: *OpenAPI Specification 3.1.0*. Linux Foundation, 2021. <https://spec.openapis.org/oas/v3.1.0>
- [7] OWASP Foundation: *OWASP API Security Top 10 2023*. <https://owasp.org/API-Security/>
- [8] S. Newman: *Building Microservices*. O'Reilly Media, 2015.
- [9] L. Richardson, M. Amundsen: *RESTful Web APIs*. O'Reilly Media, 2013.
- [10] G. Hohpe, B. Woolf: *Enterprise Integration Patterns*. Addison-Wesley, 2003.
- [11] E. Gamma, R. Helm, R. Johnson, J. Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [12] W. Vogels: *Eventually Consistent – Revisited*. ACM Queue, 6(6), 2009.